

PRO

Programmation orientée objet

Leçon I

Classes et objets

Attributs, méthodes et constructeurs

Classes et objets

Une **classe** peut être vue comme un nouveau **type de données**, un modèle, qui possède des **membres** :

- ses propres variables (attributs, champs)
- son propre comportement (méthodes ou fonctions de classe).

Par exemple : une personne, un vélo, ...

Un **objet** est une **instance d'une classe** qui a une existence propre.

Par exemple : la personne Jean Dupont, le vélo vert de Frédéric, ...

Définition d'une classe et de ses attributs

En Java, il faut créer un fichier par classe publique (par exemple `Person.java`).

Attributs :

Variables des instances de la classe.

Peuvent être de tout type (primitifs, tableaux ou classe).

Un attribut a un **niveau de protection** :

`public` pour les attributs visibles partout dans le code, peut importer la classe.

`private` pour les attributs visibles uniquement à l'intérieur de sa classe.

(voir aussi `protected` et « *package-private/default* » plus tard dans le cours)

Définition d'une classe avec des attributs, dans le fichier `Person.java` :

```
public class Person {  
    public String firstName, lastName;  
    public double salary = 1000; //valeur par défaut  
}
```

Créer et utiliser des objets

```
public class Person {  
    public String firstName, lastName;  
    public double salary = 1000; //valeur par défaut  
}
```

On crée un **objet** à l'aide du mot réservé **new**.

Par exemple : **new Person()** (les parenthèses indiquent l'exécution d'un constructeur – voir plus loin)

(La référence de) cet objet peut être assignée à une **variable** déclarée (à priori) du même type:

```
Person p = new Person();
```

La **variable** sert à accéder (référencer) à l'**objet**.

Accéder aux attributs d'un objet

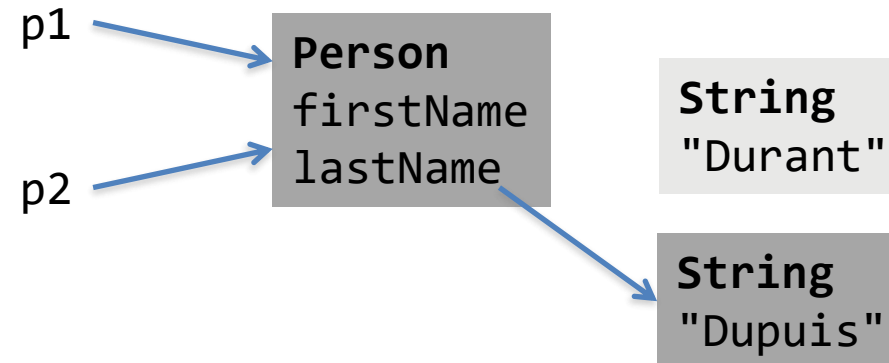
```
public class Person {  
    public String firstName, lastName;  
    public double salary = 1000; //valeur par défaut  
}
```

Une fois l'objet créé, on peut accéder à ses attributs publics, par l'intermédiaire de la variable qui le référence, à l'aide d'un **point** :

```
Person p = new Person();  
  
p.lastName = "Durant";  
p.firstName = "Jean";  
  
System.out.println(p.lastName);  
System.out.println(p.firstName);
```

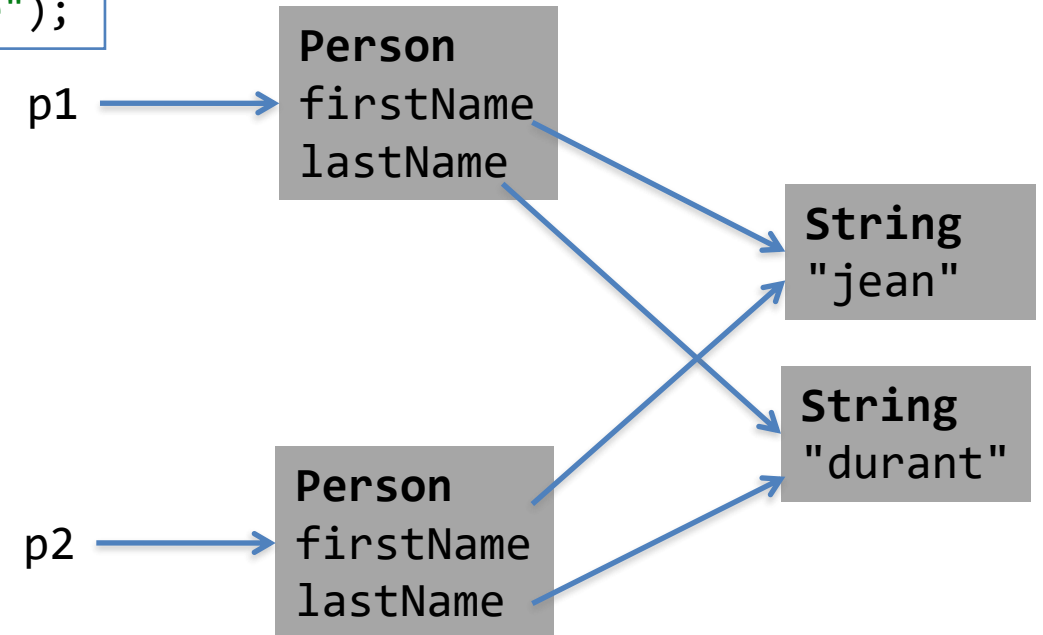
Les objets sont de type référence

```
Person p1 = new Person();  
p1.lastName = "Durant";  
  
Person p2 = p1;  
System.out.println(p2.lastName);  
  
p1.lastName = "Dupuis";  
System.out.println(p2.lastName); // !!
```



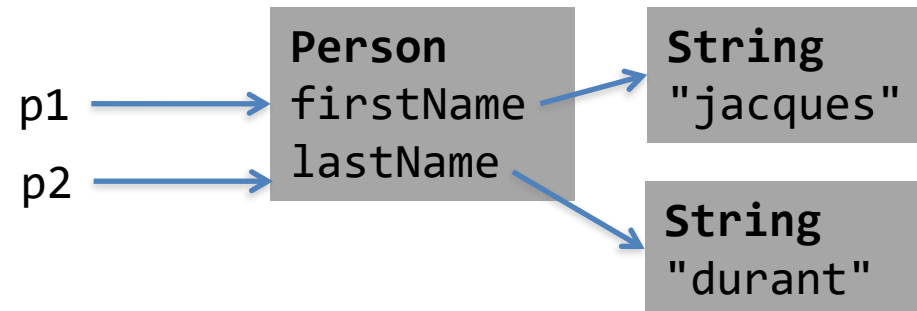
Les objets sont de type référence

```
Person p1 = new Person();  
Person p2 = new Person();  
  
p1.firstName = "jean";  
p2.firstName = p1.firstName;  
  
p1.lastName = "durant";  
p2.lastName = p1.lastName;  
  
if(p1 == p2)  
    System.out.println("Ce message n'est jamais affiché");
```



Les objets sont de type référence

```
Person p1 = new Person();  
Person p2 = new Person();  
  
p1.firstName = "jean";  
p2.firstName = p1.firstName;  
  
p1.lastName = "durant";  
p2.lastName = p1.lastName;  
  
➔ p2 = p1;  
p2.firstName = "jacques";  
  
if(p1 == p2)  
    System.out.println("Ce message est toujours affiché");
```



Définition de méthodes

Les méthodes définissent le comportement des objets de la classe.

Elles peuvent prendre ou non des paramètres et renvoyer ou non une valeur de retour.

Elles ont aussi un niveau de protection (public/private...).

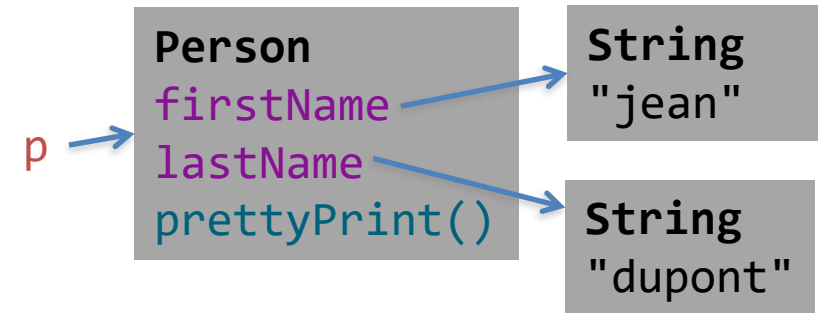
Définition d'une classe avec une **méthode** :

```
public class Person {  
    public String firstName, lastName;  
  
    public void prettyPrint(){  
        System.out.println(firstName + " " + lastName);  
    }  
}
```

Utilisation de méthodes

```
public class Person {  
    public String firstName, lastName;  
  
    public void prettyPrint(){  
        System.out.println(firstName + " " + lastName);  
    }  
}
```

```
Person p = new Person();  
p.firstName = "jean";  
p.lastName = "dupont";  
p.prettyPrint(); //affiche "jean dupont"
```



La méthode `prettyPrint` s'exécute "sur" l'objet référencé par la variable `p`.

Pendant son exécution, `p` est dit l'objet courant (`this`).

Dans le code de `prettyPrint`, l'utilisation de `firstName` et `lastName` se réfèrent aux champs de l'objet courant (`p`)

Mot clé **this**

Dans une méthode, **this** fait référence à l'objet courant :

```
public class Person {  
    public String firstName, lastName;  
  
    public void prettyPrint(){  
        System.out.println(firstName + " " + lastName);  
    }  
}
```

```
public class Person {  
    public String firstName, lastName;  
  
    public void prettyPrint(){  
        System.out.println(this.firstName + " " + this.lastName);  
    }  
}
```

Définition de constructeurs

Un **constructeur** est une méthode spéciale appelée lors de la création d'un objet.

Il sert à initialiser les objets.

Cette méthode n'a pas de type et ne renvoie rien.

Un constructeur doit avoir le même nom que la classe.

```
public class Person {  
    public String firstName, lastName;  
  
    public Person(String fn, String ln) { //permet d'initialiser l'objet  
        firstName = fn;  
        lastName = ln;  
    }  
}
```

Créer un objet

```
public class Person {  
    public String firstName, lastName;  
  
    public Person(String fn, String ln) { //permet d'initialiser l'objet  
        firstName = fn;  
        lastName = ln;  
    }  
}
```

On crée un objet à l'aide du mot réservé **new**, en indiquant le **type** de l'objet et les **arguments** permettant de l'initialiser :

```
Person jean = new Person("Jean", "Dupont");
```

Mot clé **this** et ambiguïtés

Dans une méthode et dans un constructeur, **this** fait référence à l'objet courant.

L'usage de **this** peut aussi servir quand il y a ambiguïté.

```
public class Person {  
    public String firstName, lastName;  
  
    public Person(String firstName, String lastName) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
}
```

Surcharge

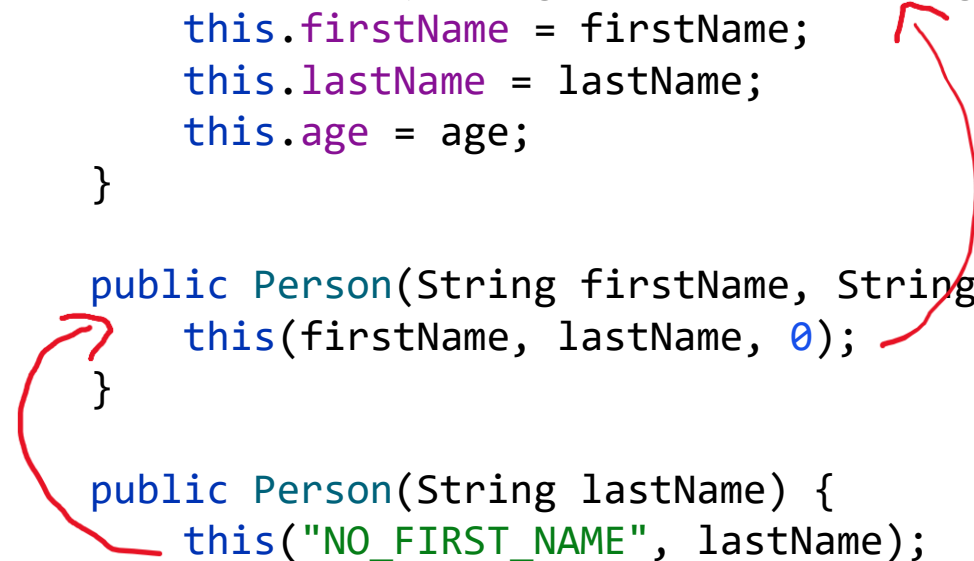
Une méthode surchargée est une méthode qui a le même nom qu'une autre mais avec des paramètres différents en nombre ou en type : en fonction de l'appel, le compilateur choisira la bonne méthode.

Il peut y avoir **plusieurs constructeurs** par classe, avec différents paramètres. On parle aussi de **surcharge**.

Dans ce contexte, `this(...)` permet de réutiliser le code d'un constructeur dans un autre.

Il ne peut se trouver qu'à la première instruction d'un constructeur.

```
public class Person {  
    public String firstName, lastName;  
    public int age;  
  
    public Person(String firstName, String lastName, int age) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
        this.age = age;  
    }  
  
    public Person(String firstName, String lastName) {  
        this(firstName, lastName, 0);  
    }  
  
    public Person(String lastName) {  
        this("NO_FIRST_NAME", lastName);  
    }  
}
```



Constructeur par défaut

Si aucun constructeur n'est défini dans une classe, le compilateur ajoute un **constructeur par défaut**. C'est à dire un constructeur **sans argument** qui ne fait (presque) **rien** - voir dans la suite.

Par contre, si un constructeur est défini explicitement, le compilateur n'ajoute pas de constructeur par défaut.

Dans ce cas, si l'on veut un constructeur sans argument, il faut l'écrire soi-même.

Appel de **constructeurs**

On crée un objet à l'aide du mot réservé **new**, en indiquant le type de l'objet et les **arguments** permettant de l'initialiser.

Ces arguments servent à sélectionner l'un des éventuels **constructeurs**.

```
public class Person {  
    public String firstName, lastName;  
    public int age;  
  
    public Person(String firstName, String lastName, int age) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
        this.age = age;  
    }  
  
    public Person(String firstName, String lastName) {  
        this(firstName, lastName, 0);  
    }  
  
    public Person(String lastName) {  
        this("NO_FIRST_NAME", lastName);  
    }  
}
```

```
Person jean = new Person("Jean", "Dupond");  
Person alain = new Person("Silovy");  
Person pierre = new Person("Pierre", "Durand", 40);  
Person johnDoe = new Person(); //erreur à la compilation
```

Conventions et bonnes pratiques

Donner des noms significatifs aux variables, méthodes, etc.

Conventions de nommages communes en Java :

lowerCamelCase pour les variables, attributs, méthodes et fonctions

UpperCamelCase pour les classes

Pour les constantes, mots en majuscules séparés par des '_'

Style d'indentation (quatre espaces)

Placement des accolades (accolade ouvrante en fin de ligne)

Ne mélangez pas les langues et préférez utiliser l'anglais.